SAME PROBLEM · DIFFERENT INFORMATION

문자열 검색의 KMP와 Rabin-Karp 비교

주제 A 알고리즘 동작 시각화

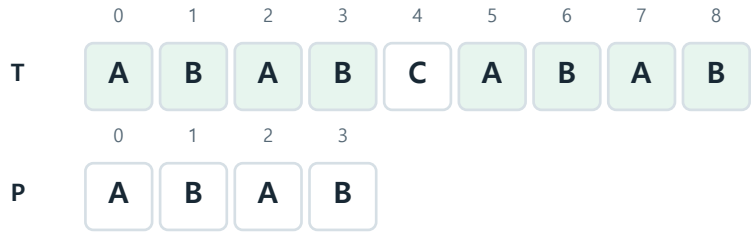
학번 20241706 이름 최종빈 분반 라

시각화 프로젝트 STRINGS LAB

1 같은 문자열에서 모든 위치 찾기

본문 문자열 T에서 패턴 P가 연속해서 나타나는 **모든 시작 위치**를 찾는다. KMP와 Rabin-Karp는 같은 문자열 검색 문제를 풀지만, 다음 비교를 결정하는 데 사용하는 정보가 다르다. KMP는 패턴의 접두사와 접미사 관계를, Rabin-Karp는 현재 구간의 해시값을 이용한다.

본문은 9자, 패턴은 4자로 정했다. 중간의 C가 KMP의 실패 이동을 보여주고, 앞뒤의 ABAB가 두 알고리즘의 같은 정답을 확인하게 한다. 인덱스는 모두 0부터 시작한다. 대소문자와 공백을 구분하고 겹치는 일치도 포함한다.



공통 입력 T = ABABCABAB, P = ABAB | **정답** [0, 5]

2 변수와 전처리 정보

기호	의미
n, m	본문 길이 n=9, 패턴 길이 m=4
i, j / s	KMP의 현재 본문·패턴 위치 / Rabin-Karp의 창 시작 위치
LPS[k]	P[0..k]의 가장 긴 진접두사이면서 접미사인 길이
b, q, H	해시의 밑 b=256, 나머지 q=101, 구간 해시 H

배포 대상: https://codesorting.github.io/algo_visual/ (새 파일 업로드 후 반영). 로컬에서는 웹사이트 폴더의 index.html을 연다. 본 보고서는 웹 링크 없이도 과정을 확인할 수 있도록 핵심 그림을 포함한다.

01 · KNUTH-MORRIS-PRATT

KMP: 이미 맞은 접두사를 재사용한다

KMP는 패턴 내부의 반복 구조를 LPS 배열로 준비한다. LPS는 자기 자신 전체를 제외한 접두사와 접미사 중 일치하는 최대 길이이다. ABAB의 LPS는 [0, 0, 1, 2]이다. 아래는 웹 구현의 핵심을 정리한 의사코드이며, match 결과는 시작 위치 목록에 추가한다.

```
01  LPS[0] = 0; len = 0; k = 1;
02  while (k < m) {
03    if (P[k] == P[len]) LPS[k++] = ++len;
04    else if (len > 0) len = LPS[len - 1];
05    else LPS[k++] = 0;
06  }
07  i = 0; j = 0;
08  while (i < n) {
09    if (T[i] == P[j]) {
10      i++; j++;
11      if (j == m) {
12        result.add(i - m);
13        j = LPS[j - 1];
14      }
15    } else if (j > 0) {
16      j = LPS[j - 1];
17    } else i++;
18  }
```

LPS를 만드는 규칙

1-6행: P[k]와 P[len]이 같으면 접두사 길이를 늘려 저장한다. 다르면 len이 가리키던 더 짧은 접두사 후보로 이동한다. 이때 k는 그대로 두어 같은 문자를 다시 판단한다. 더 짧은 후보도 없으면 0을 기록하고 k를 증가시킨다.

본문을 뒤로 돌리지 않는 규칙

9-10행: 문자가 같으면 i와 j를 함께 증가시킨다. **15-16행:** j개를 맞춘 뒤 다르면, 이미 맞은 부분의 접미사가 될 수 있는 가장 긴 접두사 길이 LPS[j-1]로 j만 바꾼다. 그동안 본문의 i는 그대로이므로 같은 본문 문자를 새 패턴 위치와 비교한다.

일치 뒤에도 계속 검색하기

11-13행: 패턴 전체를 맞추면 i-m을 정답에 넣고 j=LPS[m-1]로 돌아간다. 검색을 종료하지 않으므로 겹치는 일치도 찾는다. 예를 들어 T=ABABA, P=ABA이면 [0, 2]가 나온다.

02 · RABIN-KARP

Rabin-Karp: 창의 해시를 갱신한다

길이 m인 창을 본문에서 한 칸씩 이동한다. 창의 해시가 패턴 해시와 다르면 문자를 직접 비교하지 않는다. 같을 때만 실제 문자를 확인한다. 해시가 같아도 문자열은 다를 수 있으므로, **이 구현은 문자 검증을 통과한 경우만 정답**으로 기록한다.

```
01  hP = 0; hT = 0; high = 1;
02  for (k = 1; k < m; k++) high = high * b % q;
03  for (k = 0; k < m; k++) {
04    hP = (hP * b + code(P[k])) % q;
05    hT = (hT * b + code(T[k])) % q;
06  }
07  for (s = 0; s <= n - m; s++) {
08    if (hT == hP) {
09      j = 0;
10      while (j < m && T[s+j] == P[j]) j++;
11      if (j == m) result.add(s);
12    }
13    if (s < n - m) {
14      hT = mod(hT - code(T[s]) * high, q);
15      hT = (hT * b + code(T[s+m])) % q;
16    }
17  }
```

해시를 정하는 방법

1-6행: code(c)는 문자의 유니코드 코드 포인트이다. 영문 예시에서 A=65, B=66, C=67이다. H(x)의 각 문자는 b의 거듭제곱을 가중치로 더한 뒤 q로 나눈다. high=b^(m-1) mod q를 미리 구하면 창의 맨 앞 문자가 기여한 값을 바로 제거할 수 있다.

```
H(x[0..m-1]) = sum(k=0..m-1, code(x[k]) × b^(m-1-k)) mod q
H(next) = ((H(current) - code(out) × high) × b + code(in)) mod q
```

작은 해시값의 한계와 문자 검증

8-11행: 해시 일치는 검증할 후보를 고르는 조건이다. **14-15행:** 이전 해시에서 빠지는 문자를 빼고 새 문자를 더한다. mod(x,q)는 음수도 0..q-1로 정규화한다. 이 예시는 재현을 위해 q를 고정하며, 충돌 예시에서는 q=5로 줄인다. 무작위 소수 선택을 사용한 성능 보장을 이 고정값 실험에 그대로 적용하지 않는다.

COST MODEL

시간·공간 복잡도를 같은 조건으로 비교하기

n은 본문 길이, m은 패턴 길이이며 1 ≤ m ≤ n으로 두고 모든 일치를 찾는다. v는 해시값이 같아 문자 검증을 시작한 창의 수이다. 진짜 일치와 해시 충돌을 모두 포함한다. 아래 시간은 문자 접근·비교와 고정 크기 정수 연산을 O(1)로 보는 모형이다.

기준	KMP · LPS 방식	Rabin-Karp · 문자 검증 포함
전처리	O(m) · LPS 작성	O(m) · 최초 해시와 high
검색 시간	O(n)	O(n + v·m) 상한
전체 시간	O(n+m) · 최악에도 선형	O(n+m+v·m) 상한 최악 O(nm)
핵심 작업 공간	O(m) · LPS	O(1) · 해시·인덱스 변수
정확성	모든 문자 일치를 정확히 판단	해시 일치 뒤 문자 검증으로 보장

KMP가 선형인 이유

본문 위치 i는 감소하지 않는다. 일치하면 i와 j가 증가하고, 실패 이동은 j를 더 작은 값으로 줄인다. j의 증가 횟수는 i의 증가에 묶이므로 실패 이동도 누적해서 O(n)이다. LPS를 만들 때도 같은 방식으로 길이를 늘리거나 줄이므로 O(m)이다. 따라서 전처리와 검색을 합쳐 O(n+m)이다.

Rabin-Karp가 항상 선형이라고 할 수 없는 이유

창은 n-m+1개이며 해시 비교와 롤링 갱신은 창당 O(1)이다. 그러나 해시가 같은 v개 창마다 문자를 최대 m개 확인한다. 따라서 O(n+m+v·m)으로 정리할 수 있다. 검증 비용이 전체적으로 O(n+m)일 때 선형에 가깝게 동작하지만, 작은 고정 q에서는 충돌이 반복될 수 있다.

충돌이 없어도 모든 일치를 확인하는 비용이 커질 수 있다. T가 A의 반복이고 P도 A의 반복이면 거의 모든 창이 실제 일치한다. 각 창에서 m개를 검증하므로 v=n-m+1이고 최악의 경우 O(nm)이다. 첫 일치만 찾고 종료하는 분석과 이 구현의 전체 일치 검색을 구분해야 한다.

웹사이트의 저장 공간과는 구분한다

작업 공간 표는 입력과 결과 목록을 제외한 알고리즘 자체의 공간이다. 일치가 z개면 결과 목록은 두 알고리즘 모두 O(z)이다. 웹 구현은 문자 단위를 통일하려고 Array.from으로 입력을 복사해 O(n+m)을 쓰고, 이전 단계로 돌아가기 위해 상태 스냅샷도 저장한다. 이 시각화 기록은 위의 핵심 공간 복잡도에 포함하지 않는다.

단계 수나 서로 다른 종류의 연산 횟수를 합쳐 "몇 배 빠르다"라고 해석하지 않는다. 문자 비교, 해시 비교, 롤링 갱신은 수행하는 일이 다르며 애니메이션 속도도 알고리즘 실행 시간이 아니다.

VISUAL TRACE · LPS

LPS: 패턴 안의 반복 구조를 표로 저장하기

P=ABAB에서 LPS[k]는 P[0..k]의 진접두사와 접미사가 같은 최대 길이이다. 진접두사는 문자열 전체를 제외한다. 그림의 노란 칸은 현재 비교, 초록 칸은 같은 부분을 나타낸다.

k = 1, len = 0 — **B와 A는 다르다**

	0	1	2	3
P	A	B	A	B
LPS	0	0	-	-

P[1] 비교 P[0] → LPS[1] = 0

더 짧은 접두사 후보가 없으므로 LPS[1]=0을 기록한다.

↓

k = 2, len = 0 — **A와 A가 같다**

	0	1	2	3
P	A	B	A	B
LPS	0	0	1	-

P[2] 비교 P[0] → LPS[2] = 1

길이를 1로 늘려 LPS[2]=1을 저장한다. ABA의 앞 A와 뒤 A가 같다.

↓

k = 3, len = 1 — **B와 B가 같다**

	0	1	2	3
P	A	B	A	B
LPS	0	0	1	2

P[3] 비교 P[1] → LPS[3] = 2

길이를 2로 늘려 LPS[3]=2를 저장한다. ABAB의 앞 AB와 뒤 AB가 같다.

그림 1. ABAB의 LPS 작성 과정. 최종 배열은 [0, 0, 1, 2]이다.

LPS[3]=2는 "패턴 ABAB의 접미사 AB가 접두사 AB와 같다"는 뜻이다. 패턴 전체를 맞춘 뒤에도 이 길이 2를 재사용할 수 있다. LPS 값은 다음 시작 위치 자체가 아니라 **유지할 일치 길이**이다.

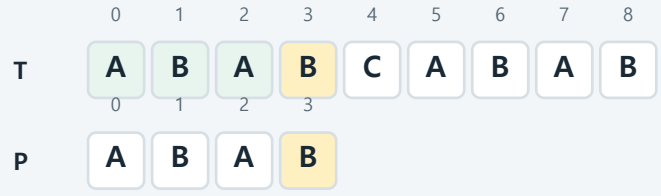
VISUAL TRACE · KMP

KMP: 같은 본문 위치에서 j만 되돌린다

첫 번째 ABAB를 찾으면 i=4이다. 전체 일치를 기록한 뒤 j=LPS[3]=2로 돌아간다. 여기서 T[4]=C와 P[2]=A를 비교하며, C를 버리기 전에 더 짧은 접두사 후보도 확인한다.

STEP 16

첫 일치: 시작 위치 0 기록

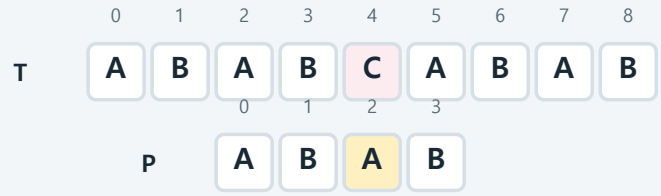


확인 문자 i=3, j=3 | 다음 i=4, j=LPS[3]=2 | result=[0]

T[0..3]이 ABAB와 같다. result=[0]. 다음 비교를 위해 j를 2로 이동한다.

STEP 19

T[4]=C와 P[2]=A 불일치

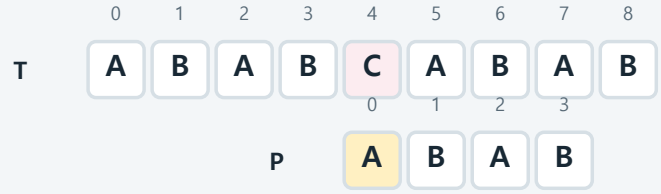


i=4, j=2 | 다음 후보 j=LPS[1]=0

이미 맞은 AB를 재사용한 후보도 C에서 실패한다. 본문 위치 i는 유지한다.

STEP 20

j만 0으로 이동

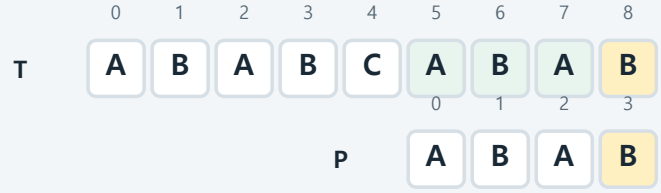


i=4 유지, j=2 → 0 | 이후 불일치하면 i=5

같은 C를 P[0]=A와 다시 비교한다. j=0에서도 다르므로 그때 i를 5로 증가시킨다.

STEP 32

두 번째 일치: 시작 위치 5 기록



확인 문자 i=8, j=3 | 다음 i=9, j=2 | result=[0,5]

T[5..8]의 ABAB를 차례로 맞춘다. 다음 i가 n=9에 도달하면 전체 검색을 마친다.

그림 2. KMP의 첫 일치 이후 실패 이동과 두 번째 일치. STEP은 웹사이트의 0부터 시작하는 실행 기록 번호이다. 실패 이동은 i를 뒤로 돌리는 것이 아니라 j를 줄여 패턴의 정렬을 바꾸는 동작이다.

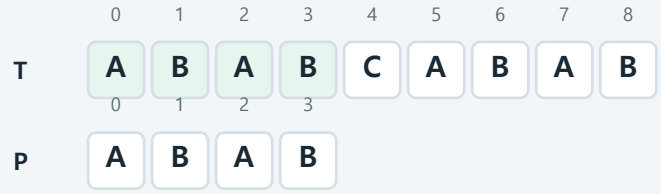
VISUAL TRACE · ROLLING HASH

Rabin-Karp: 창 전체를 다시 계산하지 않는다

b=256, q=101, m=4이면 high=256³ mod 101=5이다. 패턴 ABAB와 첫 창 ABAB의 해시는 모두 13이다. 해시가 같은 창은 문자 네 개를 확인하고, 나머지 창은 해시 차이만으로 제외한다.

STEP 6

첫 창: 해시 13 = 13

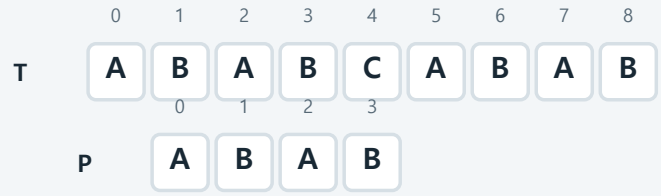


s=0, H(T[0..3])=13, H(P)=13 | 문자 검증 4회

해시 일치만으로 확정하지 않는다. A, B, A, B를 실제로 비교하여 위치 0을 기록한다.

STEP 12

ABAB → BABC: A를 빼고 C를 더하기

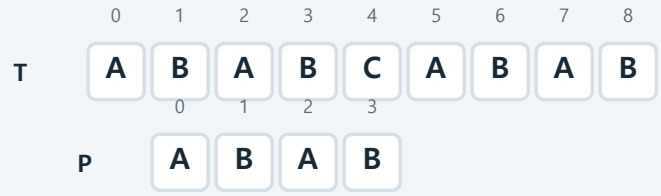


((13 - 65×5)×256 + 67) mod 101 = 86

이전 해시 13에서 A의 기여 65×5를 제거하고, 256을 곱한 뒤 C=67을 더한다.

STEP 13

두 번째 창: 해시 86 ≠ 13



s=1, H(BABC)=86, H(P)=13 | 문자 검증 없음

해시가 다르면 문자열도 다르다. BABC의 개별 문자를 패턴과 비교하지 않는다.

시작 s	0	1	2	3	4	5
창	ABAB	BABC	ABCA	BCAB	CABA	ABAB
해시 mod 101	13	86	19	5	89	13
판정	일치	제외	제외	제외	제외	일치

그림 3. 롤링 해시의 한 칸 이동과 전체 창의 판정. 마지막 s=5도 해시 일치 뒤 문자를 검증하여 정답 [0, 5]가 된다.

VISUAL TRACE · HASH COLLISION

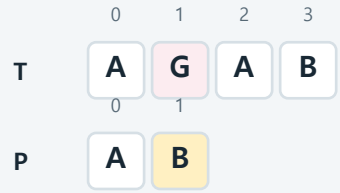
해시가 같아도 정답은 아닐 수 있다

충돌을 눈으로 확인하기 위해 별도 입력 T=AGAB, P=AB, b=256, q=5를 사용한다. 작은 q는 교육용 설정이다. A=65, B=66, G=71이므로 AG와 AB는 서로 다른 문자열이지만 해시는 둘 다 1이다.

H(AG) = (65×256 + 71) mod 5 = 1
H(AB) = (65×256 + 66) mod 5 = 1
서로 다른 문자열의 차이 71-66=5가 mod 5에서 사라진다.

STEP 6

AG와 AB: 같은 해시이므로 검증

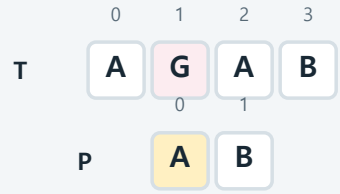


s=0, hT=1, hP=1 | 문자 검증 결과: 불일치

첫 문자 A=A는 같지만 두 번째 G와 B는 다르다. 위치 0은 정답 목록에 넣지 않는다.

STEP 11

GA도 충돌: 다시 제외한다

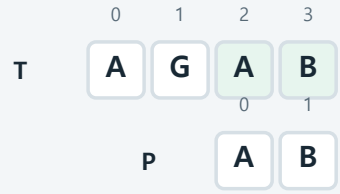


H(GA)=1=H(AB) | 두 번째 충돌도 문자 검증으로 제외

위치 0의 충돌 다음에 GA를 검사한다. 해시는 1로 같지만 첫 문자 G≠A에서 거절하여 충돌은 총 2건이다.

STEP 16

실제 AB만 정답: 시작 위치 2



s=2, hT=1, hP=1 | 문자 검증 통과

마지막 창 AB는 해시도 같고 두 문자도 모두 같다. 최종 result=[2]이고, 거짓 후보 두 개는 제외된다.

그림 4. 해시 충돌을 실제 문자 검증으로 걸러내는 과정. AG뿐 아니라 GA도 q=5에서 같은 해시가 된다. 해시가 같다는 이유만으로 반환하는 구현이라면 잘못된 위치를 정답으로 보고하게 된다.

RESULTS & REFERENCES

같은 정답, 다른 정보를 이용한 검색

기본 입력 T=ABABCABAB, P=ABAB에서 두 알고리즘은 모두 [0, 5]를 찾는다. KMP는 본문 문자를 비교하며 실패할 때 LPS로 접두사 길이를 줄인다. Rabin-Karp는 창의 해시를 비교하며 후보를 고르고, 해시가 같을 때 실제 문자를 검증한다.

기본 입력의 실제 집계	KMP	Rabin-Karp
최종 시작 위치	[0, 5]	[0, 5]
LPS 전처리 문자 비교	3	해당 없음
검색 중 문자 비교	10	8
창의 해시 비교	해당 없음	6
롤링 해시 갱신	해당 없음	5
충돌로 거절한 창	해당 없음	0

집계는 동일한 입력을 알고리즘 엔진으로 끝까지 실행한 결과이다. "검색 중 문자 비교"는 실제 문자 쌍을 비교한 횟수이고, Rabin-Karp에서는 해시 일치 후보의 검증 횟수이다. 서로 다른 종류의 작업을 단순 합산하거나 실행 속도 배율로 환산하지 않는다.

결과를 어떻게 해석할까

KMP의 장점은 패턴 구조를 이용해 최악의 경우에도 선형 시간을 유지한다는 점이다. Rabin-Karp는 해시가 다른 창의 문자 검증을 생략하고 롤링 해시로 창 이동 비용을 줄인다. 다만 충돌과 실제 일치가 많으면 검증 비용이 커진다. 이번 시각화의 핵심 비교는 "누가 항상 빠른가"가 아니라, 다음 후보를 고를 때 **무슨 정보를 재사용하는가**이다.

참고 자료 및 제작 범위

[1] Knuth, D. E., Morris, J. H. Jr., & Pratt, V. R. (1977). Fast Pattern Matching in Strings. SIAM Journal on Computing, 6(2), 323-350. https://doi.org/10.1137/0206024

[2] Karp, R. M., & Rabin, M. O. (1987). Efficient randomized pattern-matching algorithms. IBM Journal of Research and Development, 31(2), 249-260. https://doi.org/10.1147/rd.312.0249

[3] Sedgewick, R., & Wayne, K. Algorithms, 4th Edition, §5.3 Substring Search 및 강의 자료. https://algs4.cs.princeton.edu/53substring/ (확인: 2026-11-10)

[4] 첨부 웹사이트의 algorithms.js 실행 기록. 본문의 수치, LPS와 단계 그림은 이 구현에 맞춰 작성했다. 기존 QUEENS LAB은 사이트 구성과 보고서 형식의 참고로만 사용했다.

AI 활용: 코드·웹 UI 구현, 보고서 초안과 단계 그림 구성, 알고리즘 검증에 AI 도구의 도움을 받았다. 본문은 생성한 구현과 실행 기록을 기준으로 정리했다. 제출 전에는 인적 사항, 의사코드와 단계별 이동 이유를 직접 확인하여 설명할 수 있어야 한다.